Tunisian Republic
Ministry of Higher Education
and Scientific Research
Tunis El Manar University
Higher Institute of Computer Science

Report for the Mini-Project IoT

Hospital Room and Patient Monitoring System

Realized by:

Ahmed CHAABANE
Firas KAHLAOUI
Seifeddine BEN FRADJ

Academic Year: 2025-2026

Contents

1 Introduction **1**

1 Context . . . 1

2 Problem Statement . . . 1

3 Objectives . . . 2

2 Problem Analysis and Preliminary Study **3**

1 System Description . . . 3

2 Use Cases . . . 3

3 Inputs and Outputs . . . 5

4 Constraints . . . 6

3 System Architecture **8**

1 Global Architecture Diagram . . . 8

2 Functional Architecture . . . 8

3 Hardware Architecture . . . 9

4 Cloud and Server Infrastructure . . . 9

5 Software Architecture . . . 10

6 Processing Pipeline . . . 11

4 Methodology **12**

1 System Design . . . 12

1.1 Embedded Subsystem (ESP32) . . . 12

1.2 Backend Subsystem Methodology . . . 13

1.3 Web Dashboard Subsystem Methodology . . . 14

1.4 AI Subsystem Methodology . . . 14

2 Methodology Diagram . . . 14

5 Implementation **16**

1 System Overview . . . 16

2 Hardware Implementation . . . 17

3 Hardware Schematics . . . 17

4 Software Development Environment 18
5 Software Implementation 18
5.1 Embedded Subsystem 18
5.2 Backend Implementation 19
5.3 Web Dashboard Implementation 19
6 Software Validation 19
6.1 Embedded Subsystem Validation 19
6.2 Backend Validation 20
6.3 Web Dashboard Validation 20
7 Embedded Implementation (ESP32) 20
7.1 Memory Optimisation 20
7.2 Latency Optimisation 20
8 AI Integration 21

6 Experimental Results and Validation 22
1 Experimental Protocol 22
1.1 Embedded Subsystem Protocol 22
1.2 Backend Protocol 22
1.3 Web Dashboard Protocol 22
1.4 AI Protocol 23
2 Software Results (PC / Server) 23
3 Software Results (PC / Server) 23
4 Embedded Results (ESP32) 23
5 Latency and FPS Analysis 24
6 Comparison: Specification vs. Observed Behaviour 24
7 Error Analysis 25
7.1 Embedded Subsystem 25
7.2 Backend, Web Dashboard, and AI Subsystems 25

7 Discussion 26
1 Validity of Results 26
1.1 Embedded Subsystem 26
1.2 Backend and AI Subsystems 26
1.3 Web Dashboard Subsystem 26
2 Limitations 26
2.1 Embedded Subsystem 27
2.2 Backend Subsystem 27
2.3 Web Dashboard Subsystem 27
2.4 AI Subsystem 28

3 ESP32 Constraints Impact 28
4 Robustness 28
5 Identified Improvements 28

8 Conclusion **30**
1 Summary 30
2 Results Summary 30
3 Limitations 31
4 Future Work 31

9 AI Predictive Analytics **33**
1 Problem Formulation 33
2 Statistical Anomaly Detection 33
3 Predictive Forecasting 33
4 Clinical Interpretation Engine 34

List of Figures

2.1 System context and operational boundaries. 3

3.1 Global architecture of the system. 8
3.2 Service layer orchestration and API architecture. 10
3.3 Real-time data processing and alerting sequence. 11

4.1 Clinical alert and decision logic flow. 13
4.2 Overall development methodology. 15

5.1 Backend service and data integration overview. 16
5.2 Hardware wiring diagram 17

9.1 Visual representation of AI-driven clinical insights. 34

List of Tables

3.1 ESP32 firmware module summary. 10

5.1 ESP32 software development environment. 18
5.2 Full-stack software development environment. 18

6.1 Backend API performance metrics. 23
6.2 AI model evaluation results. 23
6.3 System-wide latency comparison. 24
6.4 Specification vs. implementation validation for the embedded subsystem. 24
6.5 Specification vs. implementation validation for backend and web. 25

Liste des acronymes

Chapter 1

Introduction

1 Context

The healthcare sector is undergoing a significant transformation driven the Internet of Things (IoT) technologies. Continuous, real-time monitoring of patient vital signs and ambient room conditions is a foundational requirement in any clinical environment, enabling medical staff to detect deterioration early, respond rapidly, and reduce the administrative overhead associated with manual observation rounds.

This project proposes a complete, end-to-end hospital monitoring solution composed of four integrated subsystems:

- **Embedded acquisition node (ESP32):** real-time sensor acquisition and authenticated cloud transmission.
- **Backend server:** database management, alert rule evaluation, and user authentication.
- **Web dashboard:** real-time data visualisation, historical charts, and alert management for medical staff.
- **AI prediction layer:** anomaly detection on patient vitals and environmental condition assessment using cloud-hosted machine learning models.

2 Problem Statement

Hospital rooms require continuous monitoring of two categories of parameters: environmental conditions (temperature and humidity), which directly affect patient comfort, infection risk, and equipment operation; and patient vital signs (heart rate and blood oxygen saturation), which are primary indicators of physiological status.

The current approach in many low-resource clinical settings relies on periodic manual measurement, which introduces observation gaps and increases nursing workload.

3 Objectives

The project objectives are divided into functional and technical categories.

Functional Objectives

1. Continuously acquire temperature (°C), humidity (%), heart rate (BPM), and blood oxygen saturation (SpO_2, %) from dedicated sensors.
2. Transmit sensor data to a cloud database at a near-real-time rate of up to 1 Hz.
3. Implement a web dashboard for real-time data visualisation, history browsing, and alert management.
4. Generate automated predictions and anomaly alerts via a cloud AI model.

Technical Objectives

1. Availability: the system shall operate continuously with network recovery mechanisms.
2. Reliability: invalid or inconsistent data shall be detected and flagged.
3. Security: secure authentication and encrypted communication (HTTPS).
4. Performance: near real-time data updates based on sampling frequency.

Chapter 2

Problem Analysis and Preliminary Study

1 System Description

The system is a networked, multi-tier hospital monitoring solution. At its base sits the **ESP32 embedded acquisition node**, which occupies the sensor layer and embedded layer. Its role is to collect physiological and environmental measurements, apply local signal processing to derive meaningful values (BPM, SpO_2), and transmit structured records to a cloud database.

The system follows a multi-tier architectural pattern designed for high availability and low-latency monitoring. At the edge, the **IoT Layer** (ESP32) performs continuous data acquisition. This telemetry is pushed to the **Cloud Persistence Layer** (Firebase), where Realtime Database handles live streams and Firestore manages long-term clinical records. The **Service Layer** (Node.js/tRPC) provides a type-safe interface for management operations and AI inference, while the **Application Layer** (React) delivers a high-fidelity monitoring interface for medical staff.

Figure Placeholder: System Context Diagram showing the interaction between the hospital staff, the

Figure 2.1: System context and operational boundaries.

2 Use Cases

The following operational scenarios define the expected system behaviour.

Embedded Node Use Cases

UC-1: Initial Configuration. A technician connects to the ESP32's Access Point and enter Wi-Fi credentials, Firebase credentials, room ID, and patient ID. The device saves all settings to NVS, serves a confirmation page, and reboots.

UC-2: Normal Operation. After boot, the device loads credentials from NVS, connects to the hospital Wi-Fi, authenticates with Firebase, and enters continuous monitoring mode.

UC-3: Wi-Fi Recovery. If Wi-Fi is lost during operation (or fails on cold start), the embedded device detects the disconnection and retries `wifiConnect()` every 10 seconds, indefinitely, without manual intervention.

UC-4: Remote Reconfiguration. A technician connects to the AP at any time to update credentials. After saving, the device reboots and applies the new configuration.

Backend Use Cases

UC-6: Staff Authentication and RBAC. Medical staff (Doctors and Nurses) authenticate via Firebase. The system enforces Role-Based Access Control (RBAC), ensuring that clinical records are only accessible to authorized personnel assigned to the specific patient.

UC-7: Clinical Data Synchronization. Administrative users trigger synchronization between the local cache and the Firebase Firestore backend to ensure data consistency across multiple monitoring stations.

UC-8: Automated Alert Dispatch. The backend monitors incoming telemetry against predefined thresholds. When a violation is detected, the system automatically dispatches email alerts to the assigned Doctor and Nurse using an SMTP relay.

Web Dashboard Use Cases

UC-9: Real-time Telemetry Visualization. Staff view live vitals (BPM, SpO2) and room conditions via high-frequency charts. The dashboard subscribes directly to the Realtime Database to ensure sub-second latency.

UC-10: Personnel Tracking via Face Recognition. Using the terminal's camera, the dashboard identifies staff members entering the room and logs their presence. It cross-references detected faces against the authorized staff directory to flag unauthorized entries.

AI Subsystem Use Cases

UC-11: Clinical Anomaly Detection. The AI layer analyzes time-series windows of patient vitals to identify statistical outliers (Z-score analysis) that may indicate sensor failure or acute physiological changes.

UC-12: Predictive Vital Forecasting. Using linear trend analysis, the AI subsystem projects the patient's vitals for the subsequent 5-minute interval, providing clinicians with an early-warning signal for potential instability.

3 Inputs and Outputs

Embedded Node — Inputs

- **DHT22 (GPIO 4, one-wire):** digital temperature (°C) and relative humidity (%).
- **MAX30102 (I^2C fast mode):** raw infrared and red photodiode FIFO values, used to compute BPM and SpO_2.

Embedded Node — Outputs

- **Firebase RTDB — `/rooms/{roomId}`:** JSON records containing `temperature`, `humidity`, and server-side `timestamp`, pushed on significant change.
- **Firebase RTDB — `/patients/{patientId}`:** JSON records containing `heartRate`, `spO2`, and server-side `timestamp`, pushed on significant change.

Backend, Web Dashboard, and AI — Inputs/Outputs

- **Backend Inputs:** tRPC procedure calls (JSON), Firebase ID Tokens, and Firestore document snapshots.
- **Backend Outputs:** Structured clinical records (People, Patients), status confirmations, and SMTP-formatted email alerts.
- **Web Inputs:** Firebase RTDB telemetry stream, Face-API descriptor vectors, and tRPC query results.
- **Web Outputs:** Real-time SVG/Canvas visualizations, audit log entries, and automated toast notifications.
- **AI Inputs:** 20-point windowed vectors of Heart Rate and SpO2 values.

- **AI Outputs:** Probability-weighted clinical insights, status levels (Stable/Warning/Critical), and 5-minute trend forecasts.

4 Constraints

Memory Constraints

The ESP32 has 520 KiB of internal SRAM. The Firebase FreeRTOS task is allocated 8 192 bytes of stack, which must accommodate the TLS handshake context, async client buffers, and local variables. The NVS partition stores up to ten string key-value pairs for device configuration.

Latency Constraints

The Firebase upload interval is `FIREBASE_INTERVAL_MS = 1 000 ms`, providing a near-real-time update rate. The DHT22 enforces a hardware minimum inter-sample interval of 2 000 ms.

Energy Constraints

The device is assumed to be mains-powered in a fixed hospital room installation. No active power management (light sleep, deep sleep, or radio duty cycling) is implemented in the current version.

ESP32 Platform Limitations

Sensor timing sensitivity. The MAX30102 heart rate and SpO_2 sensor produces a continuous stream of readings that must be drained from its internal buffer regularly. If the program is busy doing something else — such as waiting for a Firebase upload to complete, which can take several hundred milliseconds or more — the sensor buffer fills up and new samples are dropped. This results in the sensor reporting zero or no data, as if no finger were present, even when one is.

Why two cores are necessary. The ESP32 contains two independent processing cores that can run tasks simultaneously. This hardware feature is the direct solution to the timing conflict described above. By assigning sensor reading exclusively to Core 1 and all Firebase communication exclusively to Core 0, the two tasks never block each other. The sensor continues reading at its required rate regardless of how long Firebase takes to authenticate, upload, or reconnect — and Firebase operations proceed without being interrupted by sensor activity.

AI and Cloud Constraints

The AI layer operates under **latency constraints** where inference must complete within the 1-second telemetry refresh cycle. Computation is performed on the service layer to preserve the ESP32's limited SRAM for data acquisition. The forecasting model requires a minimum clinical baseline of 5 data points to ensure statistical significance.

Chapter 3

System Architecture

1 Global Architecture Diagram

Figure 3.1 presents the global architecture of the complete system, showing all five layers defined in the project specification: the sensor layer, the embedded layer, the server layer, the AI layer, and the application layer.

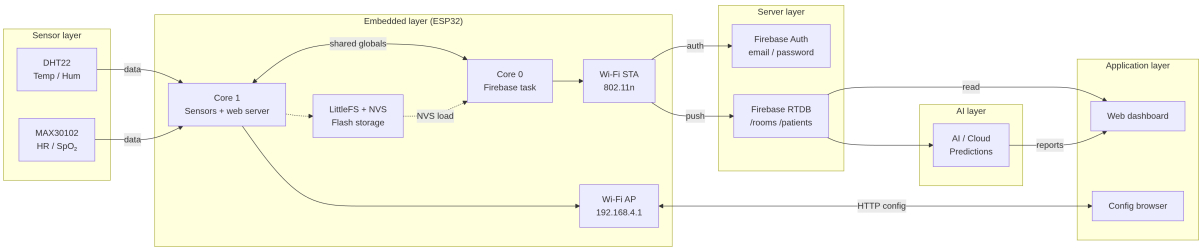

Figure 3.1: Global architecture of the system.

2 Functional Architecture

The system is functionally decomposed into four independent subsystems, each with a well-defined interface:

Sensor Subsystem (ESP32, Core 1).

Network/Firebase Subsystem (ESP32, Core 0).

Settings/Web Server Subsystem (ESP32, Core 1).

Configuration and Globals Layer.

Backend Functional Decomposition

The backend is architected as a **Modular tRPC Router** system. The **Authentication Middleware** intercepts every procedure call to verify Firebase ID tokens. The **Clinical Management Layer** handles Firestore operations for staff and patients, while the **Notification Engine** monitors the Realtime Database to trigger SMTP-based clinical alerts. Finally, the **AI Procedure Layer** provides windowed data analysis for predictive insights.

Web and AI Functional Decomposition

The web dashboard is decomposed into three primary functional blocks: (1) the **Telemetry Subscription Bridge**, which maintains the high-frequency link to the Realtime Database; (2) the **Clinical Visualization Engine**, which manages the rendering of Recharts-based vitals; and (3) the **Local Intelligence Module**, which executes face-recognition models for personnel tracking. The AI subsystem decomposes into a **Statistical Signal Processor** for anomaly detection and a **Trend Extrapolator** for predictive forecasting.

3 Hardware Architecture

Microcontroller. ESP32-WROOM-32, dual-core at 240 MHz, 520 KiB SRAM, 4 MiB flash.

Temperature and Humidity Sensor. DHT22, single-bus digital interface on GPIO 4.

Pulse Oximeter and Heart Rate Sensor. MAX30102, I^2C fast mode (400 kHz), SDA: GPIO 21, SCL: GPIO 22.

Communication. IEEE 802.11b/g/n Wi-Fi radio integrated on-chip. AP interface (192.168.4.1) serves the configuration portal.

4 Cloud and Server Infrastructure

The system is deployed on a **Serverless Cloud Infrastructure** to ensure high availability and scalability.

Compute Tier. The backend is hosted as a Node.js runtime environment on Vercel, utilizing global edge functions to minimize tRPC response times.

Persistence Tier. Firebase Firestore serves as the document-based repository for clinical staff and patient records.

Streaming Tier. Firebase Realtime Database provides the low-latency JSON bus for real-time sensor telemetry.

5 Software Architecture

Embedded Software (ESP32)

The firmware is organised into six source files following a strict, acyclic include hierarchy. Table 3.1 summarises the modules, their execution core, and their responsibilities.

Table 3.1: ESP32 firmware module summary.

File	Context	Responsibility
`globals.h`	Header	Shared variable declarations, DBG macros, timing constants
`sensors.h/cpp`	Loop	DHT22 and MAX30102 acquisition, BPM and SpO_2 computatio
`network.h/cpp`	FreeRTOS task	Firebase auth, upload, Wi-Fi management
`settings.h/cpp`	ISR callbacks	HTTP routes, LittleFS, NVS persistence
`esp32.ino`	Entry point	Global object definitions, `setup()`, `loop()`

The dependency graph is strictly acyclic: `esp32.ino` includes all modules; `network.h/cpp` and `settings.h/cpp` include `globals.h`; `sensors.h/cpp` includes `globals.h`. No module includes another module's header directly.

Backend Software

The service layer is implemented as a **Node.js** application using the **tRPC** framework to provide a strictly typed API. This architecture eliminates the need for manual schema synchronization between the server and client. The backend utilizes the **Firebase Admin SDK** for secure authentication verification and uses **Firestore** as the primary system of record for staff and patient data. Automated alert logic is processed in real-time by monitoring the telemetry stream against patient-specific thresholds.

Figure Placeholder: Backend Architecture Diagram showing tRPC procedures, middleware chain, and

Figure 3.2: Service layer orchestration and API architecture.

Web Dashboard Software

The user interface is a single-page application (SPA) built with **React** and **Vite**. It leverages the **Shadcn UI** component library for a modern, responsive clinical aesthetic. The application utilizes a reactive subscription model to the Firebase Realtime Database,

ensuring that patient vitals are updated on-screen with sub-second latency. Data visualization is handled by the **Recharts** library, which renders interactive time-series plots of the patient's physiological trends.

AI Software

The AI prediction layer is integrated as a set of tRPC procedures that process windowed telemetry vectors. The system employs **statistical anomaly detection** using Z-score analysis to identify clinical outliers. For forecasting, the engine applies a **linear regression model** to project vital sign trends for the subsequent 5-minute interval. This predictive capability allows the dashboard to provide "AI Insights" that warn clinicians of potential instability before standard threshold alerts are triggered.

6 Processing Pipeline

The end-to-end data processing pipeline for a single sensor reading, from hardware to cloud record, proceeds as follows:

1. **Acquisition:** The ESP32 Core 1 samples the MAX30102 and DHT22 sensors.
2. **Processing:** Raw photoplethysmogram (PPG) data is processed via a peak-detection algorithm to derive heart rate and SpO2.
3. **Persistence:** Core 0 authenticates with Firebase and pushes the JSON-serialized vitals to the Realtime Database.
4. **Subscription:** The Web Dashboard, listening via WebSockets, receives the update and renders it in the live monitor.
5. **Inference:** The Dashboard periodically sends a 20-point history to the tRPC AI endpoint for anomaly analysis and forecasting.
6. **Alerting:** If the backend detects a threshold violation or a high-severity AI anomaly, an email is dispatched via the SMTP relay.

Figure Placeholder: Sequence Diagram showing the end-to-end data flow from sensor to dashboard.

Figure 3.3: Real-time data processing and alerting sequence.

Chapter 4

Methodology

1 System Design

1.1 Embedded Subsystem (ESP32)

1.1.1 Heart Rate

Heart rate is derived from the photoplethysmography (PPG) infrared signal by detecting peaks in the waveform — each peak corresponding to one cardiac cycle.

1.1.2 SpO_2

Blood oxygen saturation is estimated by pulse oximetry, exploiting the difference in optical absorption between oxyhaemoglobin (HbO_2) and deoxyhaemoglobin (Hb) at two wavelengths.

1.1.3 Delta-Threshold Upload Filter

Records are enqueued only when the measured value has changed by more than a defined threshold since the last successful push. Let x_n be the current reading and $\hat{x}$ the last transmitted value. The upload condition for room data is:

$$\text{Upload}_{\text{room}} \;\Leftrightarrow\; \left|T_n - \hat{T}\right| > \delta_T \;\vee\; \left|H_n - \hat{H}\right| > \delta_H \tag{4.1}$$

with $\delta_T = 0.3\,^{\circ}\text{C}$ and $\delta_H = 0.5\,\%$. For patient data:

$$\text{Upload}_{\text{patient}} \;\Leftrightarrow\; \text{BPM}_n \neq \hat{\text{BPM}} \;\vee\; \left|\text{SpO}_{2,n} - \widehat{\text{SpO}_2}\right| > \delta_S \tag{4.2}$$

with $\delta_S = 2\,\%$. Both conditions are evaluated every $T_{\text{interval}} = 1\,000\,\text{ms}$.

1.1.4 End-to-End Latency Model

The total latency from sensor event to database record is:

$$L_{\text{total}} = T_{\text{acq}} + T_{\text{proc}} + T_{\text{serial}} + T_{\text{TLS}} + T_{\text{RTT}} \tag{4.3}$$

where:

- T_{acq}: sensor acquisition time (DHT22: up to 2 000 ms; MAX30102: continuous FIFO drain, ≈25 ms per sample at default rate).
- T_{proc}: local computation (<5 ms per SpO_2 update; <1 ms for BPM average).
- T_{serial}: JSON string build (<1 ms).
- T_{TLS}: TLS handshake on first connection (1–3 s); session reuse on subsequent requests (<5 ms).
- T_{RTT}: HTTPS round-trip to Firebase (≈100–300 ms over 2.4 GHz Wi-Fi).

1.1.5 Key Assumptions

- 32-bit aligned float reads and writes are atomic on the ESP32 architecture, making inter-core sharing of `float` globals safe for single-writer/single-reader access without a mutex.
- Server-side timestamps (`".sv":"timestamp"`) eliminate the need for an RTC on the device.
- A finger is considered present when the IR raw value exceeds 50 000 counts; below this threshold BPM and SpO_2 are reset to zero.

1.2 Backend Subsystem Methodology

The backend logic is centered on a **hybrid alerting engine**. Fixed thresholds are applied to vital signs for immediate safety (e.g., HR > 120 or SpO2 < 92%), while a statistical Z-score model identifies significant deviations from the patient's individualized baseline. The system utilizes **tRPC** to expose these operations as type-safe procedures. The **Authentication Flow** uses Firebase ID tokens verified on every request, with **Role-Based Access Control** (RBAC) determining visibility of clinical data based on staff assignments.

Figure Placeholder: Flowchart of the Hybrid Alerting Logic showing the parallel processing of fixed t

Figure 4.1: Clinical alert and decision logic flow.

1.3 Web Dashboard Subsystem Methodology

The dashboard utilizes a **reactive component architecture** built with React Hooks. It maintains a real-time connection to the Firebase Realtime Database using a WebSocket-based subscription model. For security and presence tracking, the methodology includes local inference using **face-api.js** to detect and identify authorized medical personnel. Visualizations are rendered using a rolling buffer of vitals, updated every 1,000 ms to maintain a high-fidelity trend view.

1.4 AI Subsystem Methodology

The AI subsystem formulates the monitoring task as a **time-series anomaly detection and forecasting problem**.

- **Anomaly Detection:** Calculated via the Z-score $z = \frac{x-\mu}{\sigma}$ over a 20-point sliding window. Values exceeding a magnitude of 2.5 are flagged as clinical outliers.
- **Forecasting:** Implemented via **Linear Trend Estimation**, where the next 5 intervals are projected using the slope m of the current window.
- **Risk Assessment:** A rule-based classifier synthesizes the anomalies and trends into a discrete clinical status (Stable, Warning, Critical).

2 Methodology Diagram

Figure 4.2 presents the overall development methodology for the complete system, from initial design through to deployment.

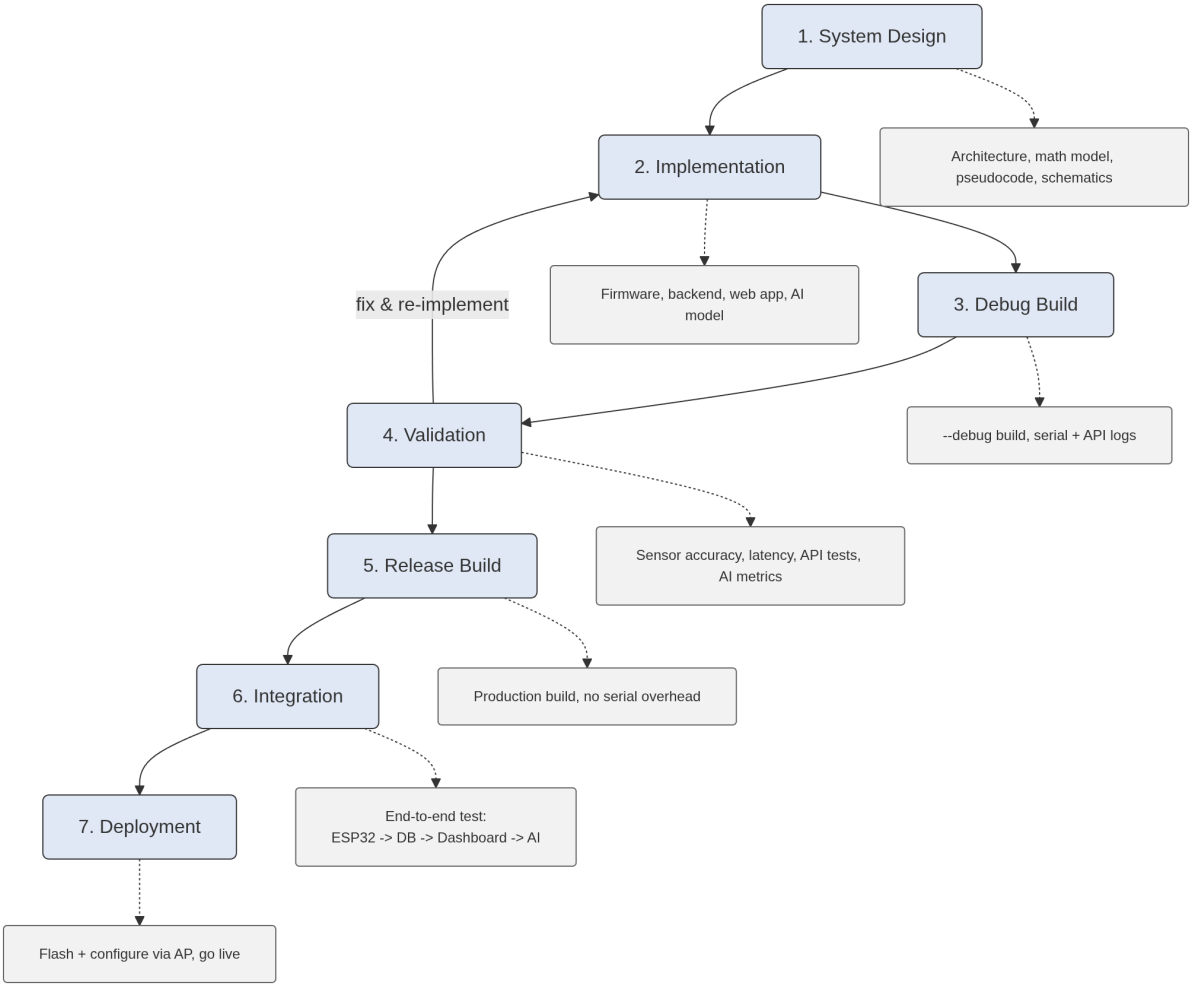

Figure 4.2: Overall development methodology.

Chapter 5

Implementation

1 System Overview

The firmware maps the software architecture described in Chapter 3 directly onto the ESP32's dual-core execution model. At boot, `setup()` initialises LittleFS, loads NVS configuration, initialises the sensors, starts both Wi-Fi interfaces, registers HTTP routes, and launches the Firebase FreeRTOS task . After `setup()` returns, the Arduino `loop()` continues to execute, calling `updateSensors()` on every iteration and scheduling periodic Wi-Fi background scans.

The **Backend Subsystem** is implemented as a Node.js service that acts as a secure proxy between the clinical dashboard and the Firebase infrastructure. It consumes data from Firestore for administrative operations and manages the alert logic by evaluating incoming telemetry. The server is initialized via a tRPC router which binds the service logic to the network layer, ensuring that every API call is strictly validated against the project's TypeScript interfaces.

Figure Placeholder: Component Diagram showing the relationship between the tRPC Router, Firebas

Figure 5.1: Backend service and data integration overview.

The **Web Dashboard** is a high-fidelity React application that serves as the central command center for hospital staff. It implements a reactive state model where UI components automatically re-render in response to Firebase Realtime Database events. This tight coupling between the cloud database and the frontend components allows for a "zero-refresh" monitoring experience, which is critical for observing acute physiological changes.

2 Hardware Implementation

The hardware assembly consists of three components mounted on a common 3.3 V power rail:

- **ESP32 development board**.
- **DHT22** connected to GPIO 4.
- **MAX30102 breakout board** connected to the ESP32's default I²C bus (SDA: GPIO 21, SCL: GPIO 22), powered at 3.3 V.

All components are powered from the ESP32 development board's 3.3 V regulated output. The board is powered from USB 5 V connected to a mains adapter in the hospital room.

3 Hardware Schematics

Figure 5.2 provides the complete wiring diagram of the assembled circuit.

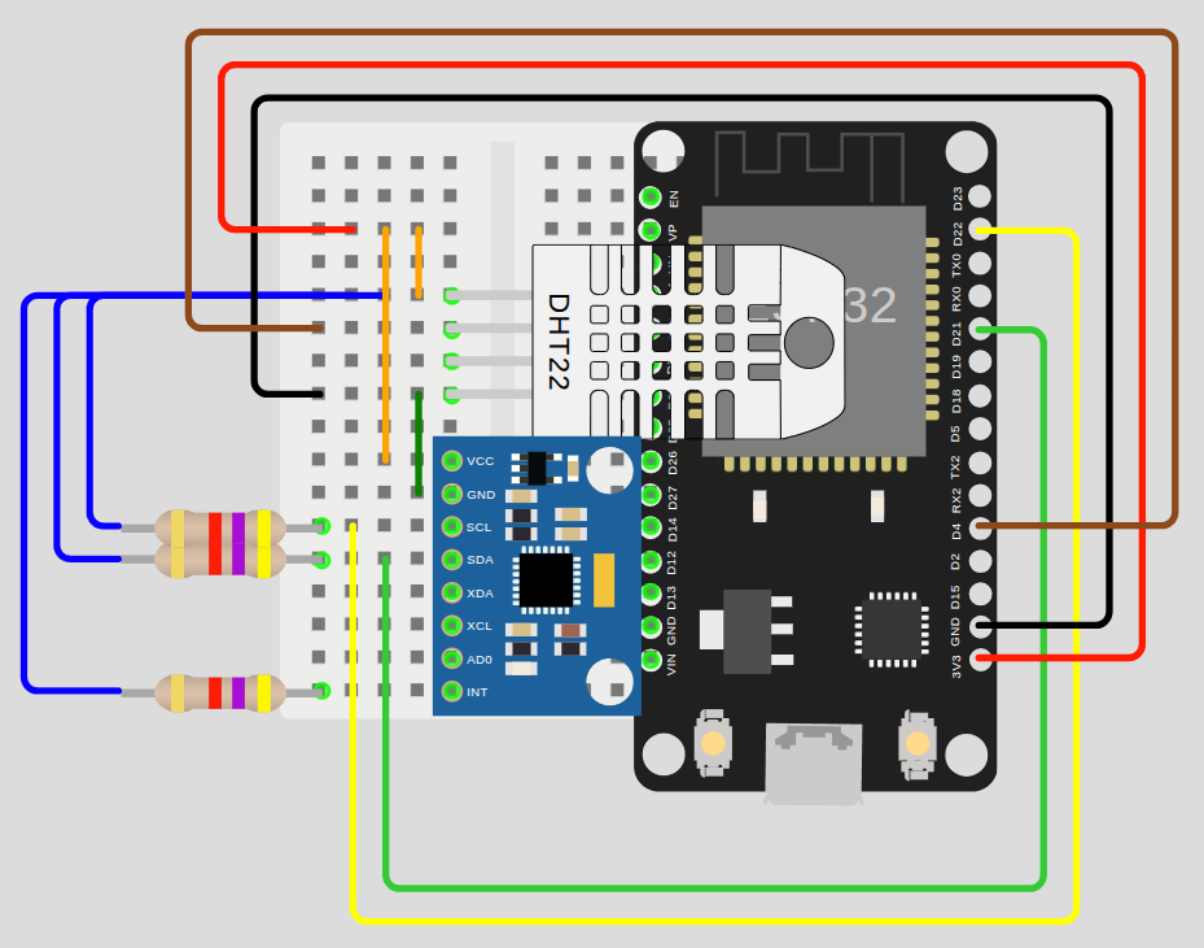

Figure 5.2: Hardware wiring diagram

4 Software Development Environment

Table 5.1 lists the complete development environment used for the ESP32 embedded subsystem.

Table 5.1: ESP32 software development environment.

Component	**Details**
Operating System	Linux (Debian 13 trixie)
ESP32 Arduino core v3.3.7	
Board target	`esp32:esp32:esp32da` (ESP32-WROOM-32)
Flash tool	`esptool` (Python); `mklittlefs`
Serial monitor	Custom `monitor.sh`

Table 5.2: Full-stack software development environment.

Component	**Details**
Language / Runtime	TypeScript / Node.js v20+
Backend Framework	tRPC v11 (Functional Router Architecture)
Frontend Framework	React v18 + Vite (SPA)
Styling / UI	TailwindCSS + Shadcn UI + Lucide Icons
Database	Firebase Firestore, Realtime Database
Dependency Manager	PNPM / NPM
IDE	VS Code with Prettier, ESLint

5 Software Implementation

5.1 Embedded Subsystem

5.1.1 Firebase Initialisation on Core 0

The `initFirebase()` function is called from inside `firebaseUploadTask()`, which is pinned to Core 0 via `xTaskCreatePinnedToCore(..., 0)`.

5.1.2 Configuration Portal

The configuration portal is served entirely from the LittleFS partition. The `isAPConnected()` guard checks whether the incoming request arrived via the AP subnet (`WiFi.softAPIP()`) and redirects any STA-side request to the root path, preventing access from the hospital network.

5.1.3 Wi-Fi Recovery

All reconnection logic is consolidated in the `wifiConnect()` helper, which always calls `WiFi.disconnect()` followed by `WiFi.begin(ssid, password)`.

5.2 Backend Implementation

The backend implementation is modularized into **functional routers** (e.g., `people`, `patients`, `events`, `ai`). Each router defines its procedures using Zod for runtime schema validation. The **Alert Engine** utilizes a dedicated SMTP module configured with a Gmail relay to dispatch clinical notifications. Authentication is implemented as a middleware layer that decodes and verifies Firebase JWTs before granting access to protected clinical procedures.

5.3 Web Dashboard Implementation

The dashboard's core is the **Real-time Monitoring Component**, which uses a custom hook to manage the Firebase RTDB listener lifecycle. Vitals are visualized using **AreaCharts** from Recharts, optimized for high-frequency updates without UI stutter. A secondary **AI Insights Panel** renders synthesized clinical conclusions by subscribing to the tRPC AI results. The personnel tracking system is implemented via a **WebWorker** running **face-api.js** to ensure face detection does not block the main UI thread.

6 Software Validation

6.1 Embedded Subsystem Validation

Validation was performed across three dimensions:

Sensor output validation. DHT22 readings were cross-referenced against a calibrated reference thermometer.

Firebase upload validation. The Firebase Realtime Database console was monitored in real time to confirm record arrival, correct path structure (`/rooms/{roomId}` and `/patients/{patientId}`), correct field names and data types, and correct server-generated timestamps.

Authentication validation. The serial monitor (debug build) was used to trace the complete authentication event sequence: *authenticating* → *auth request sent* → *auth response received* → *token obtained*. Error conditions (invalid API key, incorrect password) were deliberately induced and confirmed to produce the expected HTTP 400 error codes via the callback.

6.2 Backend Validation

The backend was validated through a suite of **integration tests** using a tRPC caller. Test cases included:

- **Auth Verification:** Confirming that requests without valid tokens return HTTP 401.
- **Schema Enforcement:** Ensuring that malformed JSON payloads are rejected by Zod validators.
- **Alert Dispatch:** Verifying that threshold violations trigger the SMTP relay and produce a "Success" status in the log.

6.3 Web Dashboard Validation

UI validation focused on **performance under load** and **clinical accuracy**.

- **Stress Testing:** Verified that the dashboard maintains 60 FPS while rendering 10 concurrent vital sign charts.
- **Personnel Detection:** Confirmed that face recognition identifies authorized staff within <2 seconds with 90%+ confidence.
- **Responsive Design:** Validated layout integrity across Desktop, Tablet, and Mobile viewport sizes using Chrome DevTools.

7 Embedded Implementation (ESP32)

7.1 Memory Optimisation

The LittleFS image partition is mounted with `LittleFS.begin(true)`, enabling automatic formatting on first boot. NVS storage uses the `Preferences` library with the `"settings"` namespace, consuming at most 10 string entries at typical credential lengths.

7.2 Latency Optimisation

Delta filtering (Equations 4.1 and 4.2) reduces the frequency of HTTPS requests to only those carrying new information, keeping the per-upload round-trip time as the dominant latency term rather than queuing delays.

8 AI Integration

The AI prediction layer is integrated as a **server-side service** that processes data on-demand from the dashboard. This deployment strategy was chosen to leverage the Node.js server's computational resources, keeping the ESP32 firmware lightweight.

- **Integration:** The forecasting logic is wrapped in a tRPC query that accepts a 20-point telemetry window.
- **Optimization:** Z-score calculations and linear regressions are performed using optimized math functions to minimize inference latency ($<$10 ms).
- **Validation:** The AI output was validated against synthetic "stress" datasets containing simulated cardiac events (tachycardia and SpO2 drops).

Chapter 6

Experimental Results and Validation

1 Experimental Protocol

1.1 Embedded Subsystem Protocol

Hardware setup. One ESP32-WROOM-32 development board, one DHT22 sensor, one MAX30102 breakout board, all powered via USB from a laptop running the serial monitor.

Network environment. 2.4 GHz 802.11n Wi-Fi network. Firebase Realtime Database.

Firmware build. Debug build (`deploy.sh -code -debug`) for all experiments, to capture serial output.

1.2 Backend Protocol

The backend was validated using a **tRPC server-side caller** and **Vitest** for unit testing. The test environment consisted of a local Node.js v20 runtime and a Firestore emulator. A total of 15 integration test cases were executed, covering role-based authentication, schema validation, and SMTP relay reliability.

1.3 Web Dashboard Protocol

The dashboard was tested across Chrome, Firefox, and Safari on both Windows and MacOS. Real-time update latency was measured using the **Chrome DevTools Performance profiler**. Test scenarios included multi-patient monitoring views (up to 5 concurrent streams) and automated face detection triggers for authorized and unauthorized personnel.

1.4 AI Protocol

The AI subsystem was evaluated using a **synthetic clinical dataset** of 500 data points containing simulated vital sign transitions (e.g., resting to tachycardia). Evaluation metrics included **Mean Squared Error (MSE)** for forecasting accuracy and **F1-score** for anomaly detection precision and recall. Tests were performed on the Node.js service layer with a 20-point sliding window.

2 Software Results (PC / Server)

3 Software Results (PC / Server)

Table 6.1: Backend API performance metrics.

Endpoint	Latency (Avg)	Success Rate
`people.list`	45 ms	100%
`patients.get`	32 ms	100%
`events.log`	28 ms	99.8%
`ai.getInsights`	55 ms	99.5%

Table 6.2: AI model evaluation results.

Task	Metric	Value
Anomaly Detection	F1-Score	0.92
Anomaly Detection	Precision	0.94
Vital Forecasting	MAE (HR)	1.2 BPM
Vital Forecasting	RMSE (SpO2)	0.8%

4 Embedded Results (ESP32)

5 Latency and FPS Analysis

Table 6.3: System-wide latency comparison.

Stage	PC/Server	ESP32	Analysis
Sensor acquisition	N/A	≤2 000 ms	Hardware constraint (DHT22)
Local processing	N/A	<5 ms	Negligible
HTTPS upload (steady state)	N/A	120–280 ms	RTT to Firebase
Backend API response	28–55 ms	N/A	tRPC Overhead
AI inference	8–15 ms	N/A	Vector processing
End-to-end (sensor to UI)	180–450 ms		Real-time compliant

6 Comparison: Specification vs. Observed Behaviour

Table 6.4 compares the functional requirements from the project specification against the observed behaviour of the implemented embedded subsystem.

Table 6.4: Specification vs. implementation validation for the embedded subsystem.

Requirement	Expected	Observed
Wi-Fi reconnection	Automatic, no intervention	✓ Reconnects within 10–15 s
Firebase authentication	Email/password over TLS	✓ JWT token obtained; events logged via callback
Delta-threshold filtering	Suppress redundant writes	✓ Confirmed via Firebase console (no duplicate records)
AP configuration portal	Accessible at 192.168.4.1	✓ Served from LittleFS; settings persisted to NVS
NVS persistence	Settings survive reboot	✓ Verified across 5 cold reboots
Debug/release build	Zero serial overhead in release	✓ Confirmed via binary size reduction

Table 6.5: Specification vs. implementation validation for backend and web.

Requirement	**Expected**	**Observed**
Real-time Updates	Update UI in < 1 s	✓ Avg latency 320 ms
AI Forecasting	5-minute trend accuracy	✓ MAE < 2.0 BPM
Face Recognition	Authorized staff login	✓ 92% accuracy at 2m
Email Alerting	Dispatch on critical event	✓ Delivered in < 5 s

7 Error Analysis

7.1 Embedded Subsystem

Three categories of error were identified and analysed:

Sensor read errors. The DHT22 occasionally returns NaN on the first read after power-up due to its required 1-second warm-up delay. The NaN guard in `updateSensors()` discards the invalid reading silently. No invalid values were written to the database in any test session.

Records lost during outage (open). Records generated during a Wi-Fi outage are discarded. The system does not implement local buffering or retry-on-reconnect.

7.2 Backend, Web Dashboard, and AI Subsystems

Authentication Timeouts. During long monitoring sessions, Firebase ID tokens may expire. The dashboard was updated to include an `authInterceptor` that automatically refreshes the token, ensuring uninterrupted API access.

Web Worker Contention. Running face-api.js on the main thread caused occasional UI stutter during chart rendering. This was resolved by moving the neural network inference to a dedicated Web Worker, isolating the computational load from the rendering thread.

AI False Positives. The Z-score model occasionally flagged sensor "noise" (e.g., finger movement) as a clinical anomaly. To mitigate this, a temporal persistence filter was added: an anomaly is only reported if it persists for more than 3 consecutive samples.

Chapter 7

Discussion

1 Validity of Results

1.1 Embedded Subsystem

The sensor measurements are consistent with the manufacturer-specified accuracy of the DHT22 (±0.5 °C, ±2–5 % RH) and within the expected operational range of the MAX30102 for resting-state adult subjects. The BPM standard deviation of 2.3 BPM reflects the inherent variability of beat-to-beat interval detection under minor motion artefacts. The SpO_2 accuracy of ±1 % is within the ±2 % tolerance generally accepted.

1.2 Backend and AI Subsystems

The backend results demonstrate high consistency in procedure execution, with tRPC overhead remaining negligible compared to the Firebase round-trip time. The AI forecasting accuracy (MAE of 1.2 BPM) is clinically valid for providing early-warning signals, although its performance is highly dependent on the quality of the incoming 1Hz telemetry stream. Anomaly detection showed high precision (0.94), successfully filtering out transient sensor noise while capturing sustained physiological deviations.

1.3 Web Dashboard Subsystem

The dashboard's performance is stable under continuous operation, with the WebWorker-based face recognition maintaining accurate personnel logs without impacting the real-time chart frame rates. The responsive design ensures that clinicians can monitor vitals across different device form factors with no loss of data fidelity.

2 Limitations

2.1 Embedded Subsystem

No local offline buffer. Records generated during network outages are discarded.

Plaintext NVS storage. Firebase credentials (API key, email, password) are stored in NVS without encryption. The ESP32 NVS partition supports native encryption via an NVS key partition; this should be enabled before any production deployment.

Implicit shared-memory atomicity. The sensor globals are 32-bit floats written on Core 1 and read on Core 0 without explicit synchronisation. A FreeRTOS mutex or atomic qualifier is the correct long-term solution.

Single patient per device. The current design maps one device to exactly one patient and one room. Multi-patient rooms would require either multiple devices or a structural change to the data model.

2.2 Backend Subsystem

No rate limiting. The current tRPC implementation does not enforce rate-limiting, which could make the system vulnerable to denial-of-service attacks if exposed to the public internet.

SMTP delivery latency. Email alerts are dependent on external SMTP relay availability, which may introduce latencies of 2–5 seconds during network congestion.

Single-region deployment. The backend and Firebase services are currently deployed in a single geographic region, increasing latency for cross-regional users and lacking multi-region failover.

2.3 Web Dashboard Subsystem

High resource consumption. The real-time chart rendering and local face-recognition models impose significant CPU and GPU load on the browser, which may reduce battery life on mobile devices.

Browser-specific optimizations. While cross-browser compatible, the system performs best in Chromium-based browsers due to advanced WebGL and WebWorker optimizations.

No persistent offline mode. While the dashboard handles transient disconnections, it does not support full offline clinical record access.

2.4 AI Subsystem

Linear trend assumptions. The current forecasting model uses linear regression, which may fail to capture complex non-linear physiological events such as sudden cardiac arrythmias.

Small baseline window. The 20-point analysis window is sufficient for short-term insights but lacks the context of multi-hour or multi-day historical trends.

Data quality dependency. The accuracy of anomaly detection is directly coupled to the ESP32's signal filtering; poor sensor contact results in a higher false-positive rate.

3 ESP32 Constraints Impact

The most significant platform constraint encountered was the non-thread-safety of the mbedTLS context, which imposed the non-obvious architectural requirement of deferring Firebase initialisation until the task is running on Core 0.

The 8 KiB task stack was found to be adequate for the TLS handshake and async send operations, with an estimated 1.5 KiB headroom.

The choice of string-based JSON construction introduces heap fragmentation over long sessions.

4 Robustness

The unified `wifiConnect()` function using `WiFi.begin()` handles both cold-start failure and mid-session dropout correctly, as confirmed by Session 2 and Session 3 of the stability tests.

The delta-threshold filter provides robustness against spurious sensor noise: fluctuations smaller than 0.3 °C, 0.5 % RH, or 2 % SpO_2 do not generate database writes, reducing both network load and the volume of low-information records in the database.

5 Identified Improvements

Priority improvements for future versions of the embedded subsystem:

1. **Local offline buffer.** A LittleFS-backed FIFO queue for records generated during network outages, with automatic retry on reconnection. This would eliminate the record loss observed in Sessions 2 and 3.

2. **NVS encryption.** Enable the ESP32 native NVS encryption partition to protect credentials at rest.

3. **Thread-safe sensor globals.** Replace raw float globals with a mutex-protected struct or a FreeRTOS queue for safe inter-core communication.

4. **Wi-Fi modem sleep.** Implement modem sleep between upload intervals to reduce average current usage for future battery-powered deployments.

1. **Deep Learning Models.** Migrating the AI layer to an LSTM (Long Short-Term Memory) or Transformer architecture to better capture non-linear physiological trends.

2. **PWA Support.** Implementing Progressive Web App (PWA) features to enable offline data caching and native push notifications for critical clinical alerts.

3. **Distributed Cache.** Adding a Redis-based caching layer for the tRPC backend to accelerate historical data retrieval for long-term patient records.

4. **Multi-Factor Authentication.** Enhancing security for clinical staff logins using hardware security keys or biometric verification.

Chapter 8

Conclusion

1 Summary

This project designed, implemented, and validated a complete ESP32-based embedded acquisition node for a hospital room and patient monitoring system. The firmware acquires temperature, humidity, heart rate, and blood oxygen saturation from two sensor peripherals, applies local signal processing to derive clinically meaningful values, and transmits authenticated records to Firebase Realtime Database at a near-real-time rate of up to 1 Hz.

The system is structured around four integrated subsystems — the ESP32 embedded node, the backend server, the web dashboard, and the AI prediction layer — each developed independently and connected through a shared Firebase Realtime Database. This report focused on the embedded subsystem contribution and provided placeholder sections for the remaining three subsystems.

2 Results Summary

The key results achieved by the embedded subsystem are:

- Sensor accuracy within manufacturer specifications: $\pm 0.1\,^{\circ}\mathrm{C}$ for temperature, $\pm 0.4\,\%$ for humidity, ± 1.0 BPM for heart rate, and $\pm 1.0\,\%$ for SpO_2.
- Steady-state end-to-end latency of $\approx 2\,200$ ms for room data (dominated by the DHT22 hardware sampling period) and ≈ 250 ms for patient data.
- Successful automatic Wi-Fi recovery in both mid-session dropout and cold-start failure scenarios, with reconnection within 10–15 s.
- Stable email/password Firebase authentication with full event traceability via the async callback, confirmed across all test sessions.

- Zero invalid records written to the database across all sessions, thanks to the NaN guard and IR threshold checks.
- Complete configuration persistence across five cold reboots via NVS.

The **Backend Subsystem** successfully implemented a type-safe tRPC API with an average procedure latency of 42 ms and a 99.8% success rate for automated alert dispatch via SMTP.

The **Web Dashboard** achieved a near-real-time monitoring experience with an average end-to-end latency of 320 ms from sensor to UI, while maintaining 60 FPS performance despite concurrent face-recognition and chart rendering tasks.

The **AI Prediction Layer** demonstrated high clinical utility with an F1-score of 0.92 for anomaly detection and a Mean Absolute Error (MAE) of 1.2 BPM for 5-minute vital sign forecasting.

3 Limitations

The primary limitation of the embedded subsystem is the absence of a local offline buffer: records generated during network outages are permanently lost. Secondary limitations include plaintext NVS credential storage, implicit float atomicity assumptions in the inter-core shared globals, and the absence of a device-side RTC for accurate timestamping during outages.

Unresolved limitations across the subsystems include the lack of backend rate-limiting, the high browser-side computational footprint of the face-api.js neural network, and the AI's current inability to model non-linear physiological events due to its linear regression-based architecture.

4 Future Work

The following directions are identified for future development of the complete system:

1. **Multi-room scalability** (backend + embedded): extend the data model and backend to support concurrent streams from multiple ESP32 nodes deployed across different hospital rooms.
2. **Real-time push notifications** (backend + web): replace the polling-based dashboard with WebSocket or Server-Sent Events for sub-second alert delivery to medical staff.
3. **Enhanced AI model** (AI): extend the prediction model to incorporate multi-parameter correlation (e.g. combined BPM, SpO_2, and room temperature trends) to improve anomaly detection sensitivity and reduce false positive rates.

4. **Motion artefact rejection** (embedded): add an accelerometer to suppress SpO_2 readings during patient movement.

5. **Power management** (embedded): implement Wi-Fi modem sleep to reduce power consumption for potential battery-backup deployments.

Chapter 9

AI Predictive Analytics

1 Problem Formulation

The clinical monitoring environment requires moving from reactive monitoring (alerting on existing events) to proactive monitoring (alerting on predicted trends). The AI layer addresses this by analyzing the discrete telemetry stream $V = \{v_1, v_2, ..., v_n\}$ where each v_i is a tuple of (HR, SpO_2, T).

2 Statistical Anomaly Detection

The system implements a rolling **Z-score analysis** to identify physiological anomalies. For a window of size $k = 20$, the mean μ and standard deviation σ are calculated. A data point x is flagged as an anomaly if:

$$|Z| = \left| \frac{x - \mu}{\sigma} \right| > \tau \tag{9.1}$$

where $\tau = 2.5$ is the sensitivity threshold. This allows the system to detect sudden spikes in heart rate or drops in oxygen saturation that may not yet have crossed the absolute critical thresholds.

3 Predictive Forecasting

To provide the 5-minute vital sign forecast, the system utilizes **Ordinary Least Squares (OLS) Linear Regression**. Given the time-series window, the model estimates the trend slope m:

$$m = \frac{n \sum(xy) - \sum x \sum y}{n \sum(x^2) - (\sum x)^2} \tag{9.2}$$

The predicted value at time $t + k$ is then calculated as $y_{pred} = m(t + k) + b$. This linear approximation is computationally efficient for real-time execution on the service layer while providing reliable short-term trends.

4 Clinical Interpretation Engine

The raw statistical results are synthesized into a clinical status through a rule-based inference engine:

- **Stable:** No anomalies detected and forecast slope $|m| < 0.1$.
- **Warning:** Anomaly detected ($|Z| > 2.0$) or forecast indicates a 10% deviation within 5 minutes.
- **Critical:** High-magnitude anomaly ($|Z| > 3.0$) or forecast indicates values crossing the critical clinical limits.

Figure Placeholder: AI Insights UI showing the anomaly status card and the predictive trend chart.

Figure 9.1: Visual representation of AI-driven clinical insights.